

Lexic.txt

Lexic:

a. Special characters, representing:

Operators:

Arithmetic: + (addition), - (subtraction), * (multiplication), ^ (power), / (division), % (mod)

Relational: == (equality), < (smaller), > (greater), <= (smaller or equal), >= (greater or equal), != (inequality)

Logical: & (and), | (or), ! (not)

Assignment: =

Index: []

Separators:

{ } () ; , <space> <newline>

Reserved words:

PROGRAM, FUNCTION, METHOD, read, write, if, else, for, do, while, break, int, char, string, list, bool, true, false return, EOP

b. Identifiers:

A sequence of letters, underscores and digits; the first character is a letter; the rule is:

identifier = letter | letter{letter | underscore | digit}
letter = "a" | "b" | ... | "z" | "A" | "B" | ... | "Z"
underscore = "_"
digit = "0" | "1" | ... | "9"

c. Constants:

1. integer:

nconst = "0" | n | "-"n
n = nonzerodigit{digit}
nonzerodigit = "1" | "2" | ... | "9"

2. character:

character = 'letter' | 'digit' | ' ' | '.' | ',' | '?' | '!' | '~' | '@' | '#' | '\$' | '%' | '^' | '&' | '*' | '(' | ')' | '-' | '+' | '_' | '=' | ':' | '>' | '<'

3. string:

string = "{character}"
const = integer | character | string

Syntax.in

program = "PROGRAM{" {declaration | stmt} programEnd "}"

(* e.g:

```
PROGRAM {  
  <declarations and/or statements, in whatever order>  
  EOP;  
}
```

*)

programEnd = "EOP;"

declaration = type " " [arraydeclaration] " " identifier{" , " identifier}

(* e.g:

```

    int n, m;
    char array[255] text, paragraph;
*)

type = "bool" | "char" | "int" | "string"

arraydeclaration = "array" "[" n "]"

simpleStmt = assignstmt | iostmt | programEnd
ioStmt = "read" "(" identifier "{", " identifier" | "write" "(" identifier
{"", " identifier"
(* e.g: read(a, b, c); write(x, y, z); *)
assignStmt = identifier "=" expression

stmt = simpleStmt | structStmt
stmtList = stmt ";" | stmt ";" stmtList
(* e.g: do1; do2; *)
structStmt = cmpdStmt | ifStmt | whileStmt
cmpdStmt = "{" stmtList "}"
(* e.g: { do1; do2; } *)

ifStmt = "if" "(" condition ")" structStmt ["else structStmt"]
whileStmt = "while(" condition ")" structStmt
(* e.g:
    while(x > 2 & x < 5){ doWork; }
*)
condition = cond | cond logical cond
cond = expression relation expression

forStmt = "for" "(" [ assignStmt ] ";" [ condition ] ";" [ assignStmt ]
)" cmpdStmt
(* e.g:
    for (i = 0; i < 10; i = i + 1) {
        x = x * 2;
        y = y + x;
    }
*)

integer = nconst
expression = term { ("+" | "-") term }
term = factor { ("*" | "/" | "^" | "%") factor }
factor = "(" expression ")" | identifier | indexedIdentifier | integer
(* e.g:
    (someArray[i] + x % 2) * 5
    The whole line above is an expression.
    `someArray[i]` is a term, `x % 2` is a (composed) term in
    which `x` is a factor, `(someArray[i] + x % 2)` is a (composed) term,
    and `5` is a term (-> factor -> integer)
*)

indexedIdentifier = identifier "[" (integer | identifier) "]"
relation = "<" | ">" | "<=" | ">=" | "==" | "!="
logical = "&" | "|"

```

token.in

PROGRAM

FUNCTION

METHOD

read

write

if

else

for

do

while

break

int

char

string

list

array

bool

true

false

return

EOP

<space>

<newline>

(

)

[

]

{

}

;

,

=

==

>

<

>=

<=

!=

&

|

!

+

-

*

^

%